Ενότητα 06:
The Pipeline IV

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

CARL

1

Τεχνολογικό Πανεπιστήμιο Κύπρου | Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Outline

- Pipeline overview
- Επίδοση pipeline
- Pipeline Hazards
- Πως τα αντιμετωπίζουμε
- Instruction-level Parallelism Optimizations (Summary)

- Υλικό από Κεφάλαιο 6 του βιβλίου του μαθήματος

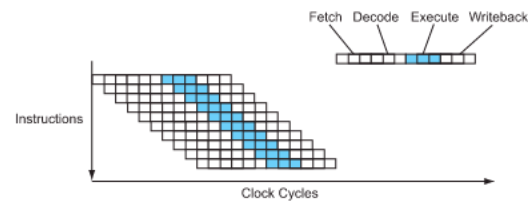
2

Advanced Pipeline Concepts

- Instruction-level parallelism (ILP)
  - Παραλληλισμός στο επίπεδο των εντολών
  - Με το Pipelining, μπορούμε να εκτελέσουμε πολλές εντολές ταυτόχρονα
- Πως μπορούμε να αυξήσουμε το ILP?
  - Βαθύτερα pipeline – Πιο πολλά στάδια (**Superpipelining**)
  - Multiple Issue (**Superscalar**)
    - Περισσότερα pipelines ανά επεξεργαστή
    - Ξεκινώ πολλές εντολές παράλληλα στον ίδιο κύκλο
    - E.g., 4GHz 4-way multiple-issue
    - 16 BIPS, peak CPI = 0.25, peak IPC = 4
  - Speculation
  - Dynamic Scheduling
  - …

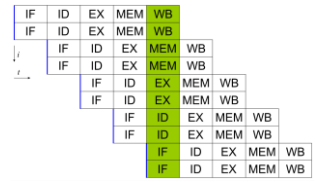
Advanced Pipeline Concepts: **Superpipelining**

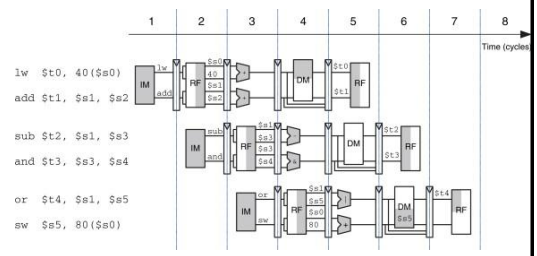
- Αυξάνουμε τον αριθμό των σταδίων, χωρίζοντας κάθε στάδιο σε μικρότερα υπο-στάδια
- Τώρα κάθε στάδιο εκτελεί πιο λίγη δουλεία που μας επιτρέπει να αυξήσουμε τη συχνότητα
- **Tradeoff:** Πιο ευαίσθητοι στους κινδύνους (hazards) και η ανάκαμψη γίνεται πιο ακριβή
- Τα σύγχρονα CPU έχουν >10 στάδια



Advanced Pipeline Concepts: **Superscalar**

- Ο επεξεργαστής μπορεί να εκτελέσει πολλές εντολές παράλληλα χρησιμοποιώντας πολλαπλά pipelines
- Διαφορετικές εντολές μπορούν να εκτελεστούν σε ξεχωριστά pipelines, <u>υπό την προϋπόθεση ότι είναι ανεξάρτητες και δεν έχουν κινδύνους</u>
- Αυξάνει το Throughput
- **Tradeoff:** Πολυπλοκότητα υλοποίησης του επεξεργαστή





Multiple Issue

- **Static multiple issue**
  - Ο Compiler ομαδοποιεί τις εντολές που μπορούν να γίνουν issue μαζί
  - Τις βάζει σε πακέτα που ονομάζονται “issue slots” ή “issue packets”
  - Ο Compiler είναι όμως υπεύθυνος να ανιχνεύει και να αποφεύγει κινδύνους (hazards)
- **Dynamic multiple issue**
  - Ο επεξεργαστής είναι υπεύθυνος για να επιλέξει ποιες εντολές θα γίνουν issue μαζί
  - Ο Compiler μπορεί να βοηθήσει (εκεί που μπορεί) με την αλλαγή της σειράς των εντολών (όταν επιτρέπεται)
  - Ο επεξεργαστής είναι όμως υπεύθυνος να ανιχνεύει και να αποφεύγει κινδύνους (hazards)

Speculation

- Μαντεύει/Υποθέτει ο επεξεργαστής τι πρέπει να κάνει με μια εντολή
  - Υποθέτει ότι όλα είναι εντάξει και Ξεκινά την εκτέλεση της άμεσα
  - Ελέγχει στο τέλος κατά πόσον ήταν σωστή η υπόθεση
    - Αν ναι, τότε όλα καλά και ολοκλήρωσε την εντολή
    - Αν όχι, τότε επανάφερε (roll-back) την εκτέλεση στη σωστή κατάσταση
- Παραδείγματα:
  - Branch εντολές/αποτέλεσμα
    - Μαντεύει το αποτέλεσμα του Branch και ακολουθά το ανάλογο μονοπάτι
    - Roll-back εάν ήταν λάθος και ακολουθεί το σωστό μονοπάτι
  - Load εντολές
    - Όταν έχουμε store → load εξάρτηση, δλδ το store αποθηκεύει στη διεύθυνση από την οποία θα πρέπει μετά να διαβάσει το load
    - Roll-back εάν πράγματι υπήρχε εξάρτηση και διάβασε τα σωστά δεδομένα

Compiler/Hardware Speculation

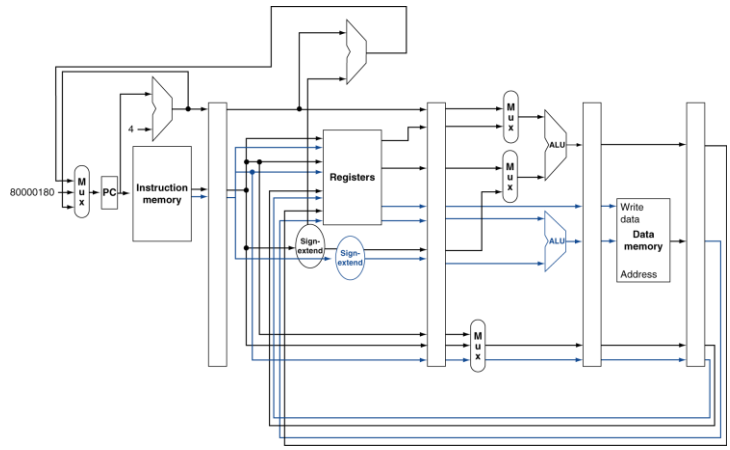
- Ο compiler μπορεί να αλλάξει την σειρά των εντολών
  - Π.χ. να μεταφέρει ένα load πριν από ένα branch
  - Μπορεί να προσθέσει και επιπλέον εντολές που να διορθώνουν τυχόν λάθη του speculation
- Ο επεξεργαστή μπορεί να κοιτάζει μπροστά (look ahead) τις εντολές για να βρει εντολές που μπορεί να εκτελέσει
  - Αποθηκεύει τα αποτελέσματα σε temporary buffers μέχρι να χρειαστούν
  - Διαγράφει (flush) τα temporary buffers εάν ήταν λάθος το speculation

MIPS με Static Dual Issue

- 2-issue slots
  - 1 ALU/branch εντολή
  - 1 load/store εντολή
  - Γιατί??

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

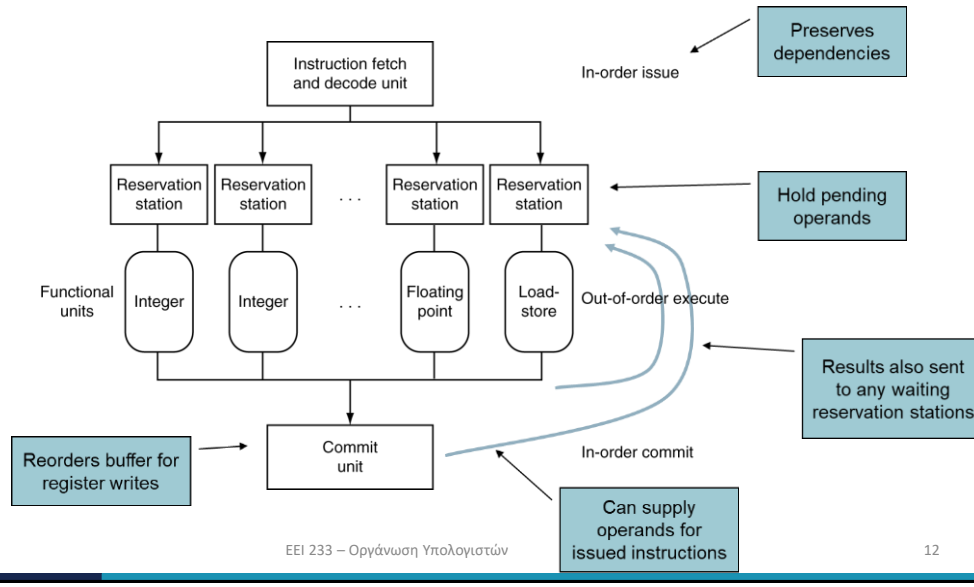
Υλοποίηση MIPS με Static Dual Issue



Does Multiple Issue Work?

- Ναι, αλλά όχι όσο θα θέλαμε
- Τα προγράμματα έχουν πραγματικές εξαρτήσεις που περιορίζουν το ILP
- Μερικές εξαρτήσεις είναι δύσκολο να τις λύσουμε
  - Π.χ., pointer aliasing
- Ο παραλληλισμός των εντολών δεν είναι πάντα εύκολος
  - Περιορισμένος αριθμός εντολών που μπορούμε να κάνουμε issue
- Καθυστερήσεις της μνήμης και περιορισμένο bandwidth
  - Καθυστερούν τα δεδομένα να έρθουν από την μνήμη
  - Δύσκολο να γεμίσουμε το pipeline

Dynamically Scheduled CPU



Why Do Dynamic Scheduling?

- Γιατί δεν αφήνουμε τον compiler να τα κάνει όλα?
  - Δεν μπορεί να προβλέψει όλα τα stalls ο compiler
    - Π.χ. Cache misses
  - Δεν μπορεί να αλλάξει πάντα την εκτέλεση των branches
    - Δυναμικό αποτέλεσμα των branches
  - Διαφορετικές υλοποιήσεις ISA έχουν διαφορετικές καθυστερήσεις και κινδύνους (hazards)
- Μια δυναμική λύση μπορεί να ανταπεξέλθει (συνήθως) καλύτερα

Ενότητα 06: The Pipeline IV

ΕΕΙ 233: ΟΡΓΑΝΩΣΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΑΝΔΡΕΑΣ ΔΙΑΒΑΣΤΟΣ

andreas.diavastos@cut.ac.cy